Turbo Pascal 5.0

```
program atari_ibm;
  uses crt,dos;
  const
    ok=0;                                {pomyslne zakonczenie operacji...}
    read_sec=2;                          {komenda dla BIOS'a}
    write_sec=3;                         {j.w.}
    size=256;                            {pojemnosc 1 sektora}
    track=18;                            {...sektorow/sciezke}

     maska:array [0..7] of byte = ($80,$40,$20,$10,8,4,2,1);
  {stale okienek}
    tt=lightred;
    pc=lightblue;
    pcmax=11;            {wysokosc okienka jak i pojemnosc bufora katalogu}
    x_pc=20;y_pc=7;
     pc_size=43;
    xl=lightmagenta;
    x_xl=14;y_xl=8;      {wsp.obszaru roboczego atari}
     xl_size=18;         {szerokosc "kursora"}
     hor=3;               {...kolumn}              {bylo odp 2 i 4}
     vert=5;             {...rzedow }
     max=hor*vert;        {tyle plikow miesci okienko}
    sp=black;
    fill=$b1;             {kod "wypelniacza" tla...}
    pc_logo:array [0..2] of array[0..6] of byte=((pc,pc,pc,sp,pc,pc,pc),
                                                 (pc,sp,pc,sp,pc,sp,sp),
                                                 (pc,sp,sp,sp,pc,pc,pc));
    xl_logo:array [0..2] of array[0..6] of byte=((sp,sp,xl,xl,xl,sp,sp),
                                                 (sp,sp,xl,xl,xl,sp,sp),
                                                 (xl,xl,sp,xl,sp,xl,xl));
    tit_logo:array [0..2] of array[0..6] of byte=((sp,sp,sp,sp,sp,tt,sp),
                                                  (sp,sp,tt,sp,tt,sp,sp),
                                                  (sp,sp,sp,tt,sp,sp,sp));

   {identyfikator dysku transferowego- BOOT loader : jmp $E471 /SELF TEST/}
    loader:array[0..8] of byte = (0,1,0,7,6,7,76,113,228);

  type
    plik = record                        {opis pliku w katalogu...}
             use:boolean;                {TRUE - plik wskazany np.do transferu}
             f_stat:byte;
             f_len,f_1st:word;
             f_name:array[1..8] of char;
             f_ext:array[1..3] of char
           end;
    sec_ptr = ^sec_log;                  {wskaznik sektora logicznego}
    sec_log = record                     {sektor logiczny w RAM}
                data:array[0..252] of byte;    {dane}
                fill:byte;                     {wypelnienie sektora}
                next:pointer                   {nastepny sektor}
              end;
    mem_buf = array [0..520] of byte; {bufor dynamiczny(min 513b+zapas)}
               {nawet rozpolowiony obszar zmiesci caly 256bajtowy sektor..}

  var
    ncom,xxx,yyy:integer;
    escape:boolean;              {wcisnieto ESC ?}
    from_pc:boolean;        {transfer z PC ?}
    select:array[0..64] of string [12]; {nazwy zaznaczonych plikow MSDOS
                                         -ostatni-nazwa pusta ''
                                         ...dlatego o 1 > niz ATARI DOS...}

    trf:file;                            {plik niezdefiniowany}
    sciezka:string;
    dir:array[0..63] of plik;            {...katalog dyskietki}
```

```
    sciezka:string;
    dir:array[0..63] of plik;          {...katalog dyskietki}
    dir2:array[0..63] of byte;         {numer rzeczywisty pliku fantomowego}
    cat:array[1..pcmax] of searchrec; {fragment katalogu dysk.MSDOS o
                                       pojemnosci okna PC}

    sec168h:word;                      {numer sektora VTOC...}
    buf,vtoc:array[0..size-1] of byte; {bufor dla 1 sektora,mapa dysku}
    buffer:^mem_buf;          {adres bufora dynamicznego...}
    buf_ofs:word;             {offset polozenia bufora fizycznego (w bloku 64kB)}
    bios_stat,d1,xdrv:byte; {status BIOS,stacja ATARI,stacja PC}

    fifo_begin,fifo_end:sec_ptr;       {pocz. i koniec kolejki}

{zmienne okienek}
    key:char;
    a:byte;
{-------------------------------------------------------------------------------}
  function any_key:char;
   var k:char;
   begin
     while keypressed do k:=readkey;
     repeat until keypressed;
     any_key:=readkey
   end;
{-------------------------------------------------------------------------------}
  function init_disk(disk:byte):byte;
    var
      r:registers;
      b_seg,b_ofs:word;
    begin
      r.dl:=disk;
      r.ah:=0;                                 {komenda:'inicjacja dysku'}
      intr($13,r);
      init_disk:=r.ah;
     {przygotuj bufor na sektor-w granicach bloku 64 kB-"proteza"...}
       new(buffer);
       b_seg:=(seg(buffer^) and $fff) shl 4;
       b_ofs:=ofs(buffer^);
       if b_seg+b_ofs<$ffff-size then buf_ofs:=0  {zmiesci sie przed $10000}
         else buf_ofs:=$ffff-(b_seg+b_ofs)+1;     {..przenies nad $ffff}
    end;
{-------------------------------------------------------------------------------}
{-Czytaj/Zapisz sektor fizyczny                                                 }

  function sector(mode,disk:byte;num:word):byte;
    var
      j:byte;
      r:registers;
    begin
      r.ah:=mode;                              {rejestr komendy}
      if mode=write_sec then for j:=0 to size-1 do buf[j]:=(buf[j] xor $ff);
      r.al:=1;                                 {ilosc sektorow}
      r.ch:=(num-1) div track;                 {...jaka sciezka?}
      r.cl:=(num-1) mod track +1;              {...jaki sektor?}
      r.dh:=0;                                 {strona 'A' dysku  (nr glowicy)}
      r.dl:=disk;                              {...jaka stacja ?}
      r.bx:=ofs(buffer^[buf_ofs]);             {adres bufora dynamicznego}
      r.es:=seg(buffer^[buf_ofs]);
      for j:=0 to size-1 do buffer^[buf_ofs+j]:=buf[j];  {...sektor do bufora dy
nam.}
      intr ($13,r);                            {BIOS - obsluga dysku}
      for j:=0 to size-1 do buf[j]:=(buffer^[buf_ofs+j] xor $ff); {...i z powrot
em z inwersem}
      sector:=r.ah;                            {status}
    end;
{-------------------------------------------------------------------------------}
{-Czytaj/Zapisz fizyczny sektor formatu Atari DOS                               }
```

```
  end;
{-------------------------------------------------------------------------}
{-Czytaj/Zapisz fizyczny sektor formatu Atari DOS                          }

  function sec_atr(mode,disk:byte;num:word):byte;
    const
       rept=3; {do 3 razy powtorz operacje z klopotliwym sektorem}
    var
       w,l,count,err:byte;
       s,o:word;
   begin
     sec_atr:=4; if (num=0) or (num>$2d0) then exit;    {nieistniejacy sektor.
..}
    {sEGMENT i oFFSET tablicy parametrow dyskietek (wektor INT $1E)}
     o:=memw[0:$1e shl 2];   s:=memw[0:($1e shl 2)+2];
    {przechowaj aktualne wart. wielkosci sektora (w) i ilosci sektorow (l)}
     w:=mem[s:o+3] ;          l:=mem[s:o+4];
     mem[s:o+3]:=1;                          {kod '1'->sektory 256bajtowe}
     mem[s:o+4]:=track;
     disk:=disk and 1;                       {...tylko FDD - A: lub B:}
     count:=0;
     repeat
      err:=sector(mode,disk,num);
      count:=count+1;
     { if err<>ok then
       if init_disk(disk)<>ok then writeln('blad inicjacji stacji ',chr(65+dis
k),':');}
     until  (err=ok) or (count=rept);
    {przywroc dawne parametry...}
     mem[s:o+3]:=w;          mem[s:o+4]:=l;
     sec_atr:=err
   end;
{-------------------------------------------------------------------------}
  function load_dir2(disk:byte):boolean;
    var
      cat,f,rel,num_file,fan:byte;
      enddir:boolean;
    begin
      load_dir2:=false;                {katalog odczytany blednie...}
      bios_stat:=sec_atr(read_sec,disk,sec168h);
      if bios_stat<>ok then exit;
      move (buf,vtoc,size); fan:=0;          {"fantomowy" nr pliku...}
      enddir:=false;
      for cat:=0 to 7 do begin
          bios_stat:=sec_atr(read_sec,disk,cat+sec168h+1);
          if bios_stat<>ok then exit;
          for f:=0 to 7 do begin
                          rel:=f*16;    { miejsce pliku w sektorze katalogu
                          num_file:=cat*8+f;  {nr kolejny pliku...}
                          with dir[num_file] do begin
                               use:=false; f_stat:=buf[rel];
                               if f_stat=0 then enddir:=true;  {koniec list
 plikow...}
                               if enddir then f_stat:=0;
                               f_len:=buf[rel+1]+$100*buf[rel+2];
                               f_ist:=buf[rel+3]+$100*buf[rel+4];
                               move(buf[rel+5],f_name,8);
                               move(buf[rel+13],f_ext,3);
                             {"czysty" katalog- odnosniki...}
                               if f_stat<>$80 then begin
                                                   dir2[fan]:=num_file;
                                                   fan:=fan+1;
                                                 end;
                          end
                        end;
      end;
     load_dir2:=true    {...przeczytany poprawnie}
   end;
{-------------------------------------------------------------------------}
```

```
  end;
{-------------------------------------------------------------------------}
  function load_dir(disk:byte):boolean;
    var
      id,no_err:boolean;
      boot,cmp,rept:byte;
      r:registers;
    begin
      r.dl:=disk; r.ah:=0; intr($13,r);              {komenda:'inicjacja stacji'
}
      rept:=1;
      repeat
        boot:=1;                  {odczytaj konfiguracje z BOOT sektorow...}
        sec168h:=$168; no_err:=true;                 {standardowy nr VTOC ? a dysk
 jest w stacji?}
        repeat
          bios_stat:=sec_atr(read_sec,disk,boot);
          if bios_stat=ok then begin              {sprawdz BOOT LOADER}
                               id:=true;
                               for cmp:=0 to 8 do if buf[cmp]<>loader[cmp] th
en id:=false;
                               if id then sec168h:=buf[$7e]+buf[$7f]*$100; {n
owy nr VTOC}
                             end
                        else no_err:=false;
          boot:=boot+1;
        until (boot>3) or (sec168h<>$168);
        if (sec168h=$168) and (no_err) then writeln('<<STANDARDOWY DYSK ATARI DO
S>>');
        load_dir:=true;
        if load_dir2(disk) then exit;
        rept:=rept+1;
      until rept>3;          {do 3 razy sztuka ....}
      fillchar(vtoc,size,0); dir[0].f_stat:=0;
      writeln('<<BRAK DYSKU TRANSFEROWEGO>>');
      load_dir:=false
    end;
{-------------------------------------------------------------------------}
  function link_sec(sector:sec_log;var act:sec_ptr):boolean;
    var
      last:sec_ptr;
    begin
      link_sec:=false;
      if maxavail<sizeof(sec_log) then exit;   {brak pamieci...}
      link_sec:=true; last:=act;
      new(act);
      move(sector.data,act^.data,253);   {przepisz 'cialo' sektora}
      act^.fill:=sector.fill;
      act^.next:=nil;
      if last<>nil then last^.next:=act
    end;
{-------------------------------------------------------------------------}
  procedure unlink_sec(var sector:sec_log;var first:sec_ptr);
    var
      act:sec_ptr;
    begin
      if first<>nil then begin
                           move(first^.data,sector.data,253);
                           sector.fill:=first^.fill;
                           act:=first^.next;
                           dispose(first);
                           first:=act
                         end
    end;
{-------------------------------------------------------------------------}
  function avail(nb:word):boolean;
    var
      bit,bajt:byte;
```

```
    var
      bit,bajt:byte;
    begin
      if nb=$2d0 then nb:=0;
      bit:=maska[nb and 7];  {nr bitu w bajcie}
      bajt:=(nb div 8)+$0a;  {nr bajtu w mapie dysku}
      avail:=((vtoc[bajt] and bit)<>0)       {true - wolny sektor nb}
    end;
{-------------------------------------------------------------------------------}
  procedure free(nb:word);
    var
      bit,bajt:byte;
    begin
      if nb=$2d0 then nb:=0;
      bit:=maska[nb and 7];
      bajt:=(nb div 8)+$0a;
      vtoc[bajt]:=vtoc[bajt] or bit
    end;

{-------------------------------------------------------------------------------}
  procedure busy(nb:word);
    var
      bit,bajt:byte;
    begin
      if nb=$2d0 then nb:=0;
      bit:=maska[nb and 7] xor $ff;
      bajt:=(nb div 8)+$0a;
      vtoc[bajt]:=vtoc[bajt] and bit
    end;

{-------------------------------------------------------------------------------}
  procedure err_msg(bios,other:byte);
    begin
      if (bios=ok) and (other=0) then exit; {pomylka....?}
      textcolor(blink+white);write(' UWAGA ! ');textcolor(white);
      if other<>0 then write('Klopoty z dyskiem MS DOS /',other,'/')
      else
      case bios of
        3:write('Dysk zabezpieczony przed zapisem...');
        4:write('Sektor niedostepny...');
        16:write('Zla suma kontrolna CRC...');
        128:write('Brak dyskietki...');
        else writeln('BIOS ERROR :',bios)
      end;
      textcolor(white); key:=any_key;
      delline;gotoxy(1,wherey)
    end;
{-------------------------------------------------------------------------------}
  function alarm(nfile,err:byte):boolean;
    var stamp:sec_log;
    begin
      alarm:=false;
      if (err=ok) and (nfile=buf[253] shr 2) then exit;
      if err<>ok then err_msg(err,0)
                 else writeln('Wadliwa struktura pliku !');
      alarm:=true;
    end;
{-------------------------------------------------------------------------------}
  function  load_atr(disk,num_file:byte):boolean;
    var
      stamp:sec_log;
      jjj,head,ln:word;
    begin
      load_atr:=false;                  {plik nie "zaladowany"}
   {czytaj plik ATARI do pamieci}
      ln:=dir[num_file].f_len;
      head:=dir[num_file].f_ist;
      repeat
        bios_stat:=sec_atr(read_sec,disk,head);
```

```
      head:=dir[num_file].f_ist;
      repeat
        bios_stat:=sec_atr(read_sec,disk,head);
        if alarm(num_file,bios_stat) or (keypressed and (ord(ReadKey)=27))
          then begin                                  {zwolnij pamiec...}
                 while fifo_begin<>nil do unlink_sec(stamp,fifo_begin);
                 exit;
               end;
        with stamp do begin           {'pieczatka' sektora }
             fill:=buf[255];
             move(buf,data,fill);
              {tescik...  for jjj:=0 to fill-1 do write(chr(buf[jjj])); writel
n}
           end;
        if not link_sec(stamp,fifo_end) then begin        {zwolnij pamiec...}
                                             while fifo_begin<>nil do unlink
_sec(stamp,fifo_begin);
                                             writeln(' <Brak pamieci dla czy
tanego pliku !> ');
                                             exit
                                           end;
        if fifo_begin=nil then fifo_begin:=fifo_end;  {1.wywolanie ->adres poc
zatku kolejki}
        head:=(buf[253] and 3)*$100+buf[254];
        ln:=ln-1;
      until (head=0) or (ln=0);
      load_atr:=true                  {plik "zaladowany"}
    end;
{-------------------------------------------------------------------------------}
  function load(disk,num_file,indx:byte):boolean;
    var
      stamp:sec_log;
      done:integer;
      io_err:byte;
    begin
      fifo_begin:=nil; fifo_end:=nil; load:=false;
      writeln(' <odczyt> ',chr(65+disk),':',dir[num_file].f_name,'.',dir[num_f
ile].f_ext);
      if not load_atr(disk,num_file) then begin
                                            select[indx]:='...';
                                            exit
                                          end;
   {zapisz buforowany plik...}
      write('   <zapis> ',select[indx]); gotoxy(1,wherey);
    {$I-}                       {wylacz kontrole IO}
      assign(trf,select[indx]);
      rewrite(trf,1);                    {1bajtowe dane}
      repeat
        unlink_sec(stamp,fifo_begin);
        blockwrite(trf,stamp.data,stamp.fill,done);
        if stamp.fill<>done then writeln('    <...plik nie zmiescil sie na dysk
u !>');
      until (fifo_begin=nil) or (stamp.fill<>done);
      close(trf);
    {$I+}                       {wlacz kontrole IO}
      io_err:=IOResult;
      if io_err<>0 then begin
                          err_msg(ok,io_err);
                          select[indx]:='...' {plik nie przeniesiony nie bedzi
e wyrozniony}
                        end
                   else begin
                          dir[num_file].use:=false;{2-giSpace=>pozostale pliki
!}
                          load:=true;
                        end;
      writeln
    end;
{-------------------------------------------------------------------------------}
```

```
{-------------------------------------------------------------------------}
  function save_dir(disk:byte;xdir:word):boolean; {fragment katalogu & VTOC}
    var dirbuf:array[0..size] of byte;
    begin
      save_dir:=false; move(buf,dirbuf,size); {przechowaj fr.katalogu}
     {najpierw zapisz vtoc...}
        move (vtoc,buf,size);                    { przenies mape do bufora...}
        bios_stat:=sec_atr(write_sec,disk,sec168h);       { ...zapisz}
        err_msg(bios_stat,0);
        if bios_stat<>ok then exit;
     {...a potem fragment  katalogu- w przypadku }
     {    nie zapisania mapy VTOC a dokonaniu wpisu w katalogu plik }
     {     pozostawia bez "parasola zajetosci" !}
      move(dirbuf,buf,size);
        bios_stat:=sec_atr(write_sec,disk,sec168h+xdir+1);
        err_msg(bios_stat,0);
        if bios_stat=ok then save_dir:=true;
   end;

{-------------------------------------------------------------------------}

  procedure erase(disk,num_file:byte);
    var
      ln,head:word;
      sec,pos:byte;
    begin
    {przejrzyj plik ATARI}
      ln:=dir[num_file].f_len;
      head:=dir[num_file].f_1st;
      repeat
        if (keypressed and (ord(ReadKey)=27)) then exit;    {ESC-zaniechaj kas
owania}
        bios_stat:=sec_atr(read_sec,disk,head);
        if alarm(num_file,bios_stat) then exit;
        free(head);                                  {"zwolnij" ten sektor ...}
        vtoc[3]:=vtoc[3]+1;                          {zwieksz liczbe sektorow}
        if vtoc[3]=0 then vtoc[4]:=vtoc[4]+1;
        head:=(buf[253] and 3)*$100+buf[254];
        ln:=ln-1;
      until (head=0) or (ln=0);
    {"skasuj" ten plik}
      sec:=num_file div 8;                       {ktory sektor katalogu?}
      pos:=(num_file mod 8)*16;                  {ktory wpis w sektorze?}
      bios_stat:=sec_atr(read_sec,disk,sec168h+sec+1);
      err_msg(bios_stat,0); if bios_stat<>ok then exit;
      buf[pos]:=$80;                             {status $80- plik skasowany}
     {zapisz fr.dir i mape VTOC}
      if save_dir(disk,sec) then dir[num_file].f_stat:=$80   { wazne dla proc
save...}
        else if (bios_stat<>3) and (bios_stat<>128)
               then begin
                      textbackground(red);textcolor(white+blink);
                      write('<<Struktura logiczna dysku prawdopodobnie uszkod
zona !>>');
                      textbackground(x1);textcolor(white); writeln;
                    end;
    end;
{-------------------------------------------------------------------------}
  function load_ibm(file_spec:string):boolean;
    var
      vint:integer;
      stamp:sec_log;
      io_err:byte;
    begin
      load_ibm:=false;
     {wczytaj plik MS DOS do pamieci jako ciag logicznych sektorow ATARI DOS}
     {$I-}                          {wylacz kontrole IO...}
      assign(trf,file_spec);
      reset(trf,1);
```

```
      assign(trf,file_spec);
      reset(trf,1);
      repeat
        if (keypressed and (ord(ReadKey)=27)) then begin    {zaniechaj ladowan
ia}
                                                    while fifo_begin<>nil do
 unlink_sec(stamp,fifo_begin);
                                                    close(trf);
                                                    exit
                                                  end;
        blockread(trf,stamp.data,253,vint);
        stamp.fill:=vint;          {vint - zmienna typu INTEGER}
        if not link_sec(stamp,fifo_end) then begin        {zwolnij pamiec...}
                                                    while fifo_begin<>nil do unlink
_sec(stamp,fifo_begin);
                                                    writeln(' <Brak pamieci dla czy
tanego pliku !> ');
                                                    close(trf);
                                                    exit
                                                  end;
        if fifo_begin=nil then fifo_begin:=fifo_end;  {1.wywolanie ->adres poc
zatku kolejki}
      until vint<>253;
      close(trf);
     {$I+}                          {...przywroc kontr. IO}
      io_err:=IOResult;
      if io_err=0 then load_ibm:=true
                  else begin
                         while fifo_begin<>nil do unlink_sec(stamp,fifo_begin
);
                         err_msg(ok,io_err);
                       end
    end;
{-------------------------------------------------------------------------------}
  function dos_name:string;
    var
      jjj,kkk:byte;
      eqstr,fspec:string;
      rpt:boolean;
      znak:char;
    begin
      repeat;
        repeat write('   <nowa nazwa: '); readln(fspec); until fspec<>'';
        jjj:=1;kkk:=1; eqstr:='            '; rpt:=false;
        repeat
            znak:=upcase(fspec[jjj]);
            if znak='.' then if jjj=1 then rpt:=true          {nazwa od kropk
i ?}
                                      else if kkk<9 then kkk:=9 else rpt:=true
;   {wiecej niz 1 kropka?}
            if (jjj=9) and (kkk=9) and (znak<>'.') then rpt:=true;     {powinn
a byc kropka a nie ma ?}
            if (znak<='Z') and (znak>='A') or (znak>='0') and (znak<='9')
    {znak nie jest dozwolony ?}
                or (znak='.') or (znak='_') then if not rpt then eqstr[kkk]:=z
nak
                                            else else rpt:=true;

            jjj:=jjj+1;kkk:=kkk+1
        until (kkk>12) or (jjj>length(fspec));   {cale pole wpisu wypelnione l
ub cala nazwa przerobiona}
      until not rpt;
      dos_name:=eqstr
    end;

{-------------------------------------------------------------------------------}
    function save (nsel:byte;file_spec:string;disk:byte):boolean;
      var
        iii,kkk,pos:byte;
```

```
{-------------------------------------------------------------------------}
  function save (nsel:byte;file_spec:string;disk:byte):boolean;
    var
      jjj,kkk,pos:byte;
      fnd,present,f_len,f_1st,f_num,free_sec,sec:word;
      fname:string [11];
      stamp:sec_log;
    begin
      save:=false;                {plik nie zostal przeniesiony...}
      fifo_begin:=nil;   fifo_end:=nil;
      writeln(' <odczyt> ',chr(xdrv+64),':',file_spec);
      if not load_ibm(file_spec) then exit;
    repeat
      fname:='           ';     {wyczysc pole nazwy...}
       kkk:=1;jjj:=1;
       repeat
         if file_spec[kkk]='.' then jjj:=8
         else fname[jjj]:=file_spec[kkk];
         jjj:=jjj+1; kkk:=kkk+1;
       until kkk>length(file_spec);
     {szukaj pliku o tej samej nazwie...}
      fnd:=0;while (dir[fnd].f_stat<>0) and (fnd<64) and
                 ( (fname<> (dir[fnd].f_name+dir[fnd].f_ext) ) or (dir[fnd].
f_stat=$80) )
               do fnd:=fnd+1;
      if (fnd<64) and (dir[fnd].f_stat<>0) then begin   {jest taki plik}
           writeln(' <plik o nazwie ',file_spec,' istnieje !>');
           writeln(' <ENTER -zapisz bez zmiany nazwy...>');
           key:=any_key;
           if ord(key)=13
            then begin
                   erase(d1,fnd);   {...najpierw usun poprzedni plik..}
                   if dir[fnd].f_stat<>$80 then exit; {nie zostal usuniety...
}
                 end
            else begin
                   fname:='';file_spec:=dos_name;
                 end
        end;
    until fname<>'';
     {szukaj wolnego miejsca w katalogu...}
      fnd:=0;while (dir[fnd].f_stat<>$80) and (dir[fnd].f_stat<>0) and (fnd<64
)
               do fnd:=fnd+1;
      if fnd=64 then begin
                       writeln(' <UWAGA Katalog jest przepelniony !>');
                       exit
                     end;
      f_num:=fnd;
     {szukaj pierwszego wolnego sektora....}
      fnd:=4; while (not avail(fnd)) and (fnd<=$2d0) do fnd:=fnd+1;
      if fnd>$2d0 then begin
                         writeln(' <UWAGA Dysk jest przepelniony !>');
                         exit
                       end;
      f_1st:=fnd; f_len:=0;
      writeln('   <zapis> ',chr(65+disk),':',file_spec);
      repeat
        present:=fnd;             {zm. fnd -FIND/FOUND...present-obecny}
        busy(present);
      {...szukaj nastepnego}
          while (not avail(fnd)) and (fnd<=$2d0) do fnd:=fnd+1;
          if fnd>$2d0 then writeln('   <...plik nie zmiescil sie na dysku !>')
;
        unlink_sec(stamp,fifo_begin);
        if (fifo_begin=nil) or (fnd>$2d0) then fnd:=0;   {ostatni sektor}
      { uformuj sektor...}
          f_len:=f_len+1;
```

```
          f_len:=f_len+1;
          move(stamp.data,buf,stamp.fill);
          buf[255]:=stamp.fill;
          buf[254]:=lo(fnd);
          buf[253]:=(f_num shl 2) or hi(fnd);
          bios_stat:=sec_atr(write_sec,disk,present);
          err_msg(bios_stat,0); if bios_stat<>ok then exit;
        until fnd=0;
      {"zapisz" ten plik}
        sec:=f_num div 8;                    {ktory sektor katalogu?}
        pos:=(f_num mod 8)*16;               {ktory wpis w sektorze?}
        bios_stat:=sec_atr(read_sec,disk,sec168h+sec+1); {odczytaj fragment kata
logu}
        err_msg(bios_stat,0); if bios_stat<>ok then exit;
        {... zmieniony fragment katalogu}
         {wstaw nowy plik...}
         buf[pos]:=$42;                                    {status pliku...}
         buf[pos+1]:=lo(f_len);buf[pos+2]:=hi(f_len);      {dlugosc pliku...}
         buf[pos+3]:=lo(f_1st);buf[pos+4]:=hi(f_1st);      {pierwszy sektor...}
                                                           {nazwa + rozsz. ...}
         for jjj:=0 to 10 do buf[pos+5+jjj]:=$20;  { wyczysc pole nazwy...}
         kkk:=1;jjj:=0;
         repeat
           if file_spec[kkk]='.' then jjj:=7
           else buf[pos+5+jjj]:=ord(file_spec[kkk]);
           jjj:=jjj+1; kkk:=kkk+1;
         until kkk>length(file_spec);  + or {until jjj>10}
       {...zmieniony VTOC}
         free_sec:=vtoc[3]+$100*vtoc[4];
         free_sec:=free_sec-f_len;
         vtoc[3]:=lo(free_sec); vtoc[4]:=hi(free_sec);  {xxx free sectors...}
       { zapisz vtoc i fr. dir...}
         if save_dir(disk,sec)
          then begin
                 save:=true;
                 str(f_num,select[nsel]);   {zapamietaj nr zapisanego pliku}
               end
          else if (bios_stat<>3) and (bios_stat<>128)
                 then begin
                        textbackground(red);textcolor(white+blink);
                        write('<<Struktura logiczna dysku prawdopodobnie uszkod
zona !>>');
                        textbackground(x1);textcolor(white); writeln;
                      end;
      end;
{=============================================================================}
    procedure wrt_bios(znak,atr:byte);
      var r:registers;
          lastx:byte;
      begin
        r.ah:=9; r.al:=znak;       {postaw 'znak' ze zmiana atrybutow}
        r.bh:=0; r.bl:=atr; r.cx:=1;          {strona 0, 1 znak}
        if (znak<>13) and (znak<>10)
          then begin
                 intr($10,r); lastx:=wherex;
                 gotoxy(lastx+1,wherey); {kursor w prawo}
                 if lastx-wherex=0 then writeln; {koniec okienka->nast.linia}
               end
          else if znak=13 then writeln;
      end;
{-----------------------------------------------------------------------------}
  procedure rd_bios(var znak:byte;var atr:byte);
      var r:registers;
      begin
        r.ah:=8; r.bh:=0; intr($10,r);     {odczyt znaku i atrybutu, nr strony}
        znak:=r.al;
        atr:=r.ah;
      end;
```

```
    end;
{-------------------------------------------------------------------------}
  procedure win_ini(txt,bkg:byte);
    var
      f:word;
      r:registers;
    begin
      window(1,1,80,25);
      textbackground(bkg);
      r.ah:=$0b;  r.bh:=0; r.bl:=bkg; intr($10,r); {ustaw kolor ramki...}
      textcolor(txt);
      clrscr;
    end;
{-------------------------------------------------------------------------}
  procedure put_logo(blk:boolean;logo:byte);
    var
     inf,x,y:byte;
    begin
      for y:=0 to 2 do
        for x:=0 to 6 do begin
                              if blk then textcolor(logo+blink) else textcolor(l
ogo-8);
                              if logo=tt then gotoxy(x+1,y+1) else gotoxy(x+72,y
+21);
                              if logo=xl then inf:=xl_logo[y,x]
                                else if logo=pc then inf:=pc_logo[y,x]
                                  else inf:=tit_logo[y,x];
                              if (not blk) and (logo<>tt) then inf:=sp
                                 else  textbackground(inf);
                              if inf<>sp then write(chr(fill))
                                else begin
                                       textbackground(lightgray);
                                       if logo<>tt then write(chr($20))
                                         else begin
                                                textcolor(darkgray);write(chr($
c5));
                                              end;
                                     end
                           end
    end;
{-------------------------------------------------------------------------}
   procedure title(wait:byte);
     var
       hr,mn,sec,sec100,loop:word;
     begin
        window(19,11,80,20); textcolor(white);
        gotoxy(1,2);                  writeln('        Program wymiany danych z
systemem Atari DOS');
        textcolor(lightgray);write('          ');   {ukryj kursor...}
        textcolor(lightred);          writeln('~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
~~~~~~~~~~');
        textcolor(white);             writeln('                              autor:  Wojc
iech Karolik');
                                      writeln('                                    O
LSZTYN 1994');
        put_logo(true,tt);
        gettime(hr,mn,sec,sec100);
        loop:=sec+wait; if loop>59 then loop:=loop-59;
        repeat gettime(hr,mn,sec,sec100) until sec=loop;
        put_logo(false,tt);
        window(1,1,80,25);
     end;
{-------------------------------------------------------------------------}
  procedure open_win(headline:string;ctxt,cbkg,lx,ly,px,py:byte);
    var
      pion,poz,znak,atr:byte;
    begin
      window(1,1,80,25);if (cbkg=xl) or (cbkg=pc) then py:=py+1;{okno PC lub XL/
XE?}
```

```
XE?}
      textbackground(black);
      textcolor(lightgray);
      for pion:=ly+1 to py+1+3 do begin
                                  gotoxy(px+1,pion);rd_bios(znak,atr);
                                  if atr=lightgray*16+white then write(chr(fill)
)
                                    else wrt_bios(znak,lightgray);
                                end;
      gotoxy(lx+1,py+1+3);
      for poz:=lx+1 to px do begin
                               rd_bios(znak,atr);
                               if atr=lightgray*16+white then write(chr(fill))
                                 else wrt_bios(znak,lightgray);
                             end;
      textbackground(cbkg);
      textcolor(ctxt);
      window(lx,ly,px,py+3);
      clrscr;
      write(chr($da));
      for poz:=lx+1 to px-1 do write(chr($c4));
      write(chr($bf)); write(chr($b3));
      gotoxy(px-lx+1,2);
      write(chr($b3)); write(chr($c0));
      for poz:=lx+1 to px-1 do write(chr($c4));
      write(chr($d9));
      gotoxy(2,2); write(headline); {wypisz naglowek}
      if (cbkg=xl) or (cbkg=pc) then py:=py-1;{...okno PC lub XL/XE}
      window(lx,ly+3,px,py+3);       {ustal okienko robocze}
    end;
{-------------------------------------------------------------------------------}
  procedure pc_free;
    var f:byte;
        info:string;
    begin
      str(diskfree(xdrv),info); info:=concat(' ',info,' BAJTOW ');
      window(x_pc,y_pc+pcmax+4,x_pc+pc_size+1,y_pc+pcmax+4); gotoxy(2,1);
      for f:=1 to pc_size-2 do write(chr($c4));
      gotoxy(pc_size-length(info)-2,1); write(info);
      window(x_pc,y_pc+3,x_pc+pc_size,y_pc+pcmax+3);
    end;
{-------------------------------------------------------------------------------}
  procedure atr_free;
    var f:byte;
        info:string;
    begin
      str(vtoc[3]+vtoc[4]*$100,info); info:=concat(' ',info,' SEKTOROW ');
      window(x_xl,y_xl+vert+4,x_xl+hor*xl_size+1,y_xl+vert+4);gotoxy(2,1);
      for f:=1 to hor*xl_size-1 do write(chr($c4));
      gotoxy(hor*xl_size-length(info)-2,1); write(info);
      window(x_xl,y_xl+3,x_xl+hor*xl_size,y_xl+vert+3);
    end;
{-------------------------------------------------------------------------------}
  procedure cursor(on:boolean;num,rel:byte);
    var
      x,y:byte;
    begin
      x:=(num mod hor)*xl_size+1;
      y:=(num div hor)+1;
      gotoxy(x,y);
      num:=num+rel;
      textcolor(white);if not on then lowvideo;
      if dir[dir2[num]].use then textcolor(yellow);
      if on then textbackground(red) else textbackground(xl);
      with dir[dir2[num]] do begin
                               if f_stat and $20<>0 then write('*') else write
(' ');
                               write(f_name,'.',f_ext,' ',f_len:3,' ');
                             end;
```

```
                        end;
      gotoxy(x,y); highvideo
    end;
{-------------------------------------------------------------------------------}
  function win_dir:integer;
    var
      df,fle,delta:integer;
    begin
     atr_free;          {free sectors...}
     win_dir:=-2;   {brak pliku do transferu...}
     if dir[dir2[0]].f_stat=0 then exit;   {katalog jest pusty...}
     fle:=0;delta:=hor;  {kruczek - od razu wyswietli 1.strone}
     repeat
       if (fle-delta>=max) or (fle<delta) then
          begin
            clrscr;
            if fle<delta then delta:=delta-hor else delta:=delta+hor;
            for df:=0 to max-1 do if (dir[dir2[df+delta]].f_stat<>0) then
              cursor(false,df,delta)
          end;
       cursor(true,fle-delta,delta);
       repeat
         escape:=true;
         key:=upcase(any_key);
           if ord(key)=80 then win_dir:=dir2[fle] {P -view...}
           else if (ord(key)=32) and (not from_pc) then win_dir:=-1  {jest kand
ydat do transferu}
                else if ord(key)<>27 then escape:=false;
         df:=1;
         if ord(key)=0 then key:=readkey;  {kod rozszerzonej klawiatury-2. bajt
}
               case ord(key) of
                 82:if not from_pc then dir[dir2[fle]].use:=true;
                 83:if not from_pc then dir[dir2[fle]].use:=false;
                 80:if dir[dir2[fle+hor]].f_stat=0 then df:=1 else df:=hor;
                 72:if fle-hor<0 then df:=-1 else df:=-hor;
                 75:df:=-1;
                 77:df:=1;
                else df:=0
               end;
       until (df<>0) or escape;
       cursor(false,fle-delta,delta);
       if (fle+df)>=0) and (dir[dir2[fle+df]].f_stat<>0) then fle:=fle+df;
     until escape;                {ESCape...}
    end;
{-------------------------------------------------------------------------------}
    procedure view_atr(disk,fnum:byte;pcview:boolean);
     const
       xw=25;yw=3;high=17;
     var
       fix,cnt:byte;
       hdl,fspec:string;
       stamp:sec_log;
     begin
       if pcview then fspec:=concat(chr(64+disk),':',cat[fnum].name);
       if pcview then hdl:=concat('Zawartosc pliku: ',fspec)
        else hdl:=concat('Zawartosc pliku: ',chr(65+disk),':',dir[fnum].f_name,
'.',dir[fnum].f_ext);
       open_win(hdl,white,cyan,xw,yw,xw+40+1,yw+high);
       window(xw+1,yw+3,xw+40,yw+3+high);  {ustal marginesy}
       fifo_begin:=nil;
       fifo_end:=nil;
       if pcview then if not load_ibm(fspec) then exit
        else else if not load_atr(disk,fnum) then exit;
       cnt:=0;                          { z linia "zerowa" wlacznie}
       repeat
         unlink_sec(stamp,fifo_begin);
         for fix:=0 to stamp.fill-1 do begin
```

```
        unlink_sec(stamp,fifo_begin);
        for fix:=0 to stamp.fill-1 do begin
          if (stamp.data[fix]=$0d) or (wherex=39) then cnt:=cnt+1;
          if cnt<=high then wrt_bios(stamp.data[fix],cyan*16+white)
          else begin
                 key:=any_key;
                 if ord(key)=27 then begin   {zwolnij pamiec i uciekaj...}
                      while fifo_begin<>nil do unlink_sec(stamp,fifo_begin);
                      exit;
                    end;
                 wrt_bios(stamp.data[fix],cyan*16+white);
                 cnt:=high;    {line down}
                 if ord(key)=0 then begin
                                      key:=readkey;
                                      if ord(key)=81 then cnt:=1    {PgDn}
                                    end
               end;
          end;
        until fifo_begin=nil;
        key:=any_key
      end;
{-------------------------------------------------------------------------------}
  procedure win_del;
    var ackn:integer;
    begin
     from_pc:=false;  {operacje z ATARI...}
     window(1,1,80,25);
     put_logo(true,x1);  gotoxy(1,25); textcolor(white);
      write(' ESC-zaniechaj:INS,DEL-selekcja plikow:P-przejrzyj (PgDn):SPACJA-sk
asuj pliki ');
     repeat
       open_win('Wybierz pliki Atari DOS do skasowania...',white,x1,x_x1,y_x1,x_
x1+hor*x1_size,y_x1+vert);
       ackn:=win_dir;
       if ackn>=0 then view_atr(d1,ackn,false);
     until ackn<0;
     if ackn=-2 then exit;          {....wyjscie przez ESC-bez reakcji}
     clrscr;textcolor(white);
     ackn:=0; while (dir[ackn].f_stat<>0) and (ackn<64) do
                begin
                  if dir[ackn].use then begin
                       writeln(' <plik:> ',chr(65+d1),':',dir[ackn].f_name,'.',
dir[ackn].f_ext);
                       write('   <czy skasowac ten plik ? (T/N) > ');
                       repeat
                        key:=any_key; key:=upcase(key);
                       until (key='T') or (key='N');
                       if key='N' then writeln('NIE')
                                  else begin
                                         writeln('TAK');
                                         if dir[ackn].f_stat and $20<>0
                                           then writeln('   <plik jest zabezpie
czony przed skasowaniem !>')
                                           else  erase(d1,ackn);
                                         if dir[ackn].f_stat<>$80
                                           then writeln('    < ...nie zostal sk
asowany ! > ');
                                       end;
                    end;
                  ackn:=ackn+1
                end;
     from_pc:=true;      {bez manipulacji na plikach...}
     if not load_dir(d1) then exit;
     repeat
       open_win('...dysk Atari DOS...',white,x1,x_x1,y_x1,x_x1+hor*x1_size,y_x1+
vert);
       ackn:=win_dir;
       if ackn>=0 then view_atr(d1,ackn,false);
     until ackn<0;
```

```
    until ackn<0;
  end;
{-------------------------------------------------------------------------------}
  function cat_buf(rel:byte):byte;
    var
      df:byte;
      info:searchrec;
    begin
      cat_buf:=0;          {nie wczytano zadnego pliku...}
      findfirst('*.*',directory+archive,info);
      df:=0;
      while (df<rel) and (doserror<>18) do begin
                                          findnext(info);
                                          df:=df+1;
                                        end;
      if doserror=18 then exit;  {nie ma wiecej plikow}
      df:=1;cat[df]:=info;
      repeat
        findnext(info);
        cat[df+1]:=info;
        if doserror<>18 then df:=df+1;
      until (doserror=18) or (df>pcmax);
      cat_buf:=df;
    end;
{-------------------------------------------------------------------------------}
  procedure line(on:boolean;ypos:byte);
    var
      birth:datetime;             {czas narodzin pliku}
      cnt,lng:byte;
    begin
      if on then textbackground(red) else textbackground(pc);
      highvideo;textcolor(white);
      if {(cat[ypos].attr<>$10) and} (not on) then lowvideo;
      xxx:=0;                              {oddziel wybrane pliki...}
      while select[xxx]<>'' do begin
         if cat[ypos].name=select[xxx] then textcolor(yellow);
         xxx:=xxx+1
       end;
      gotoxy(1,ypos);
      for cnt:=1 to pc_size+1 do write(' ');gotoxy(2,ypos); cnt:=1;
      with cat[ypos] do begin
                          lng:=length(name);
                          repeat;
                            if (name[cnt]='.') and (attr<>$10) then gotoxy(10,ypo
s);
                            write(name[cnt]);cnt:=cnt+1;
                          until cnt>lng;
                          if attr=$10 then begin          {podkatalog?}
                                             gotoxy(14,ypos);write('<DIR>')
                                           end
                                      else begin
                                             gotoxy(15,ypos);write(size:10)
                                           end;
                          unpacktime(time,birth);gotoxy(27,ypos);
                          with birth do begin
                                          write(day,'.',month,'.',year);
                                          gotoxy(38,ypos);write(hour,':',min)
                                        end
                        end;
      gotoxy(1,ypos);highvideo
    end;
{-------------------------------------------------------------------------------}
  procedure get_put(get:boolean;ypos:byte);
    var
      fnd:byte;
    begin
      if not from_pc then exit;    {...tylko gdy transfer z PC !}
      fnd:=0;if cat[ypos].attr=$10 then exit; {podkatalog ?}
```

```
  begin
    if not from_pc then exit;    {...tylko gdy transfer z PC !}
    fnd:=0;if cat[ypos].attr=$10 then exit; {podkatalog ?}
    while select[fnd]<>''  do  begin
                                 if cat[ypos].name=select[fnd] then
                                    if get then exit else select[fnd]:='...';
                                 fnd:=fnd+1
                               end;
    if get then begin
                  if fnd>63 then begin  {wybor max 64 plikow....}
                       fnd:=0;                {...szukaj "dziur"...}
                       while select[fnd]<>'...' do begin
                          if (fnd>63) then exit;  {wybor max 64 plikow....}
                          fnd:=fnd+1
                       end
                     end
                  else select[fnd+1]:='';  {...nie trzeba bylo szukac "dziur"}
                  select[fnd]:=cat[ypos].name;
                end;
    line(false,ypos);    {...wyroznij kolorem}
  end;

{-------------------------------------------------------------------------------}
 procedure look_path(drv:byte;sub_dir:string);
   var f:byte;
       path,drive:string;
   begin
     getdir(drv,path);drive:=chr(64+drv); {ascii 65-A,66-B...}
     window(x_pc+2,y_pc+2,x_pc+pc_size-3,y_pc+2); gotoxy(1,1);
     for f:=1 to pc_size-5 do write(chr($c4)); gotoxy(1,1);
     if length(path)>pc_size-5 then path:=concat(drive,':\...\',sub_dir);
     write(path);
     window(x_pc,y_pc+3,x_pc+pc_size,y_pc+pcmax+3);
   end;
{-------------------------------------------------------------------------------}
 function overwrite(num_file:byte):string;
   var
      sps:integer;
      fname:string[12];
      info:searchrec;
   begin
      fname:=dir[num_file].f_name;
      sps:=pos(' ',fname);if sps<>0 then delete(fname,sps,9-sps);
      fname:=concat(fname,'.',dir[num_file].f_ext);
      repeat
        findfirst(fname,directory+archive,info);
        if doserror=0 then begin
              writeln(' <plik ',fname,' istnieje !>');
              writeln(' <ENTER -zapisz bez zmiany nazwy...>');
              key:=any_key;
              if ord(key)<>13 then fname:=dos_name;
           end
         else if doserror<>18 then doserror:=18;
      until (ord(key)=13) or (doserror=18); {czy "nowy" tez istnieje?}
      overwrite:=fname
   end;
{-------------------------------------------------------------------------------}
 procedure xchdir(root:string);
   var io_err:byte;
   begin
     {$I-}
      chdir(root); io_err:=IOResult;
      if io_err<>0 then begin
                          clrscr;textbackground(red);
                          writeln(' <Dysk ',chr(64+xdrv),': jest niedostepny ?
!> ');
                          escape:=true;
                          select[0]:='';   {wyjscie jak przez ESC...}
                        end;
```

```
                        select[0]:='';   {wyjscie jak przez ESC...}
                      end;
    {$I+}
  end;
{-------------------------------------------------------------------------------}
 procedure win_cat(drive:byte;new_dir:string);
  var
    refresh,trans:boolean;              {"odswiezyc" okno ?}
    delta:integer;
    df,v,xxx,yyy,ord_key:byte;
  begin
    pc_free;                {free bytes...}
    xchdir(new_dir);if escape then exit;
    look_path(drive,new_dir);
    v:=0;delta:=1;refresh:=false;trans:=false;select[0]:='';
  repeat
    if (refresh) and (cat[v].attr=$10) and (cat[v].name<>'.') and (cat[v].name
<>'..')
       then begin
              win_cat(drive,cat[v].name);
              if escape then exit;     {zakoncz "wedrowke"...}
                {refresh window...}
              look_path(drive,new_dir);
              refresh:=true;
            end
       else refresh:=false;
    if (v<1) or (v>pcmax) then begin    {kursor poza okienkiem?}
                                 refresh:=true;
                                 if v<1 then begin
                                               v:=1;delta:=delta-1
                                             end
                                        else begin
                                               v:=pcmax;delta:=delta+1
                                             end;
                               end;
  {"odswiez" okno }
    if (refresh) or (trans) then begin
                      df:=cat_buf(delta);
                      if df>pcmax then yyy:=df-1 else yyy:=df;
                      clrscr;
                      for xxx:=1 to yyy do line(false,xxx);
                    end;
    refresh:=false;trans:=false;
    line(true,v);  key:=upcase(any_key);  line(false,v);
    case ord(key) of
      0: begin
           key:=readkey;ord_key:=ord(key);
           if ord_key=82 then get_put(true,v);     {Insert}
           if ord_key=83 then get_put(false,v);    {Delete}
           if (ord_key=82) or (ord_key=83) then if from_pc then
                                                  ord_key:=80; {...w
 dol}
           if ord_key=72 then if v>1 then v:=v-1
                                     else if delta>0 then v:=v-1;
           if ord_key=80 then if v<df then v:=v+1
                                      else if df>pcmax then v:=v+1;

         end;
       80:begin
            view_atr(drive,v,true);
            open_win('Wybierz pliki MS DOS...',white,pc,x_pc,y_pc,x_pc+pc_size,
y_pc+pcmax);
            pc_free;
            trans:=true;          {...i wypisz fragment katalogu w odtworzonym ok
nie}
          end;
       27:begin
            escape:=true;
            select[0]:=''         {nie wybrano zadnych plikow}
```

```
            escape:=true;
            select[0]:=''      {nie wybrano zadnych plikow}
          end;
       13:refresh:=true;       {Enter-zmien podkatalog}
       32:if not from_pc then begin          {Space-przenies pliki ATARI}
            clrscr;  trans:=true;yyy:=0;
            for xxx:=0 to $3f do
              if dir[xxx].use then begin
                                     select[yyy]:=overwrite(xxx);
                                     if not load(d1,xxx,yyy) {przenies plik nr
 xxx}
                                      then begin
                                        select[yyy]:='...';
                                        writeln(' <...plik nie zostal przenies
iony !>');
                                      end;
                                     yyy:=yyy+1;
                                   end;
            pc_free;           {...zmiany pojemnosci dysku!}
            select[yyy]:=''; {koniec listy zaznaczonych plikow}
          end
        else escape:=true ;  {Space-przenies pliki PC}
      end;
     if escape then exit;      {zakoncz "wedrowke"...}
   until (refresh) and (cat[v].name='..');
   xchdir('..');      {wroc katalog macierzysty...}
   select[0]:='';   {pliki moga byc wybrane tylko z JEDNEGO podkatalogu!}
 end;
{-------------------------------------------------------------------------------}
{---Transfer z ATARI na PCta---}

 procedure atr2pc;
   var exst:boolean;
   begin
    from_pc:=false;   {transfer z ATARI...}
    window(1,1,80,25);
    put_logo(true,x1);   gotoxy(1,25); textcolor(white);
     write('ESC-zaniechaj:INS,DEL-selekcja plikow:P-przejrzyj (PgDn):SPACJA-tra
nsfer na PC');
    repeat
     open_win('Wybierz pliki Atari DOS...',white,x1,x_x1,y_x1,x_x1+hor*xl_size
,y_x1+vert);
      xxx:=win_dir;
      if xxx>=0 then view_atr(d1,xxx,false);
    until xxx<0;
    if xxx=-2 then exit;          {....wyjscie przez ESC-bez reakcji}
    exst:=false;
    for xxx:=0 to 63 do exst:=exst or dir[xxx].use; {sa pliki do przeniesienia?
}
    if not exst then exit;        {...nie ma}
      window(1,1,80,25);
      put_logo(true,pc);    gotoxy(1,25); textcolor(white);
       write('ESC-zaniechaj:ENTER-zmiana podkatalogu:SPACJA-transfer do aktualn
ego katalogu ');
      open_win('...dysk MS DOS...',white,pc,x_pc,y_pc,x_pc+pc_size,y_pc+pcmax);
      escape:=false;
      win_cat(xdrv,chr(64+xdrv)+':\');      {katalog glowny stacji XDRV:}
  end;
{-------------------------------------------------------------------------------}
{-----Transfer z PCta na ATARI-------}
  procedure pc2atr;
    var
      exst:boolean;
      mvd,ackn:integer;
      nfle:byte;
   begin
      from_pc:=true;              {transfer z PC}
      window(1,1,80,25);
      put_logo(true,pc);    gotoxy(1,25); textcolor(white);
```

```
      put_logo(true,pc);    gotoxy(1,25); textcolor(white);
      write(' ESC-zaniechaj:ENTER-zmiana podkatalogu:INS,DEL-zaznacz pliki:SPAC
JA-transfer ');
      open_win('Wybierz pliki MS DOS...',white,pc,x_pc,y_pc,x_pc+pc_size,y_pc+p
cmax);
      escape:=false;
      win_cat(xdrv,chr(64+xdrv)+':\');   {katalog glowny stacji XDRV:}
                  {get_put() wybiera 63 pliki !!!!!}

     nfle:=0; exst:=false;
     while select[nfle]<>'' do begin
                                  exst:=(exst) or (select[nfle]<>'...');
                                  nfle:=nfle+1;
                                end;
     if not exst then exit; {nie ma zaznaczonych plikow}
     window(1,1,80,25); nfle:=0;
     put_logo(true,xl);   gotoxy(1,25); textcolor(white);
     write(' ESC-koniec:P-przejrzyj (PgDn)
           ');
     open_win('...dysk Atari DOS...',white,xl,x_xl,y_xl,x_xl+hor*xl_size,y_xl+v
ert);
     while select[nfle]<>'' do begin
                                 if select[nfle]<>'...' then
                                  if (not load_dir(d1)) or (not save (nfle,sele
ct[nfle],d1))
                                    then writeln(' <...plik nie zostal przenies
iony !>');
                                 nfle:=nfle+1
                                end;
     from_pc:=true;       {bez manipulacji na plikach...}
     if not load_dir(d1) then exit;
     nfle:=0; while select[nfle]<>''   {oznacz przeniesione pliki...}
                  do begin
                       val(select[nfle],mvd,ackn);
                       if ackn=0 then dir[mvd].use:=true;
                       nfle:=nfle+1
                     end;
     repeat
       open_win('...dysk Atari DOS...',white,xl,x_xl,y_xl,x_xl+hor*xl_size,y_x
l+vert);
       ackn:=win_dir;
       if ackn>=0 then view_atr(d1,ackn,false);
     until ackn<0;
   end;
{-------------------------------------------------------------------------------}
  procedure scan_disk(disk:byte);
    var
      t,s:byte;
    begin
      open_win('Co wybierasz ?... ',white,red,44,7,76,14);
      textcolor(yellow); write(' Scanning dysku w stacji '+chr(65+disk)+': ');
      textcolor(white); writeln('<S>');
      textcolor(yellow); write(' Przydzial stacji           ');
      textcolor(white); writeln('<P>');
      textcolor(yellow); writeln('   /',chr(65+d1),':-stacja ATARI  ',chr(64+xdr
v),':-PC/');
      key:=upcase(any_key);if (key<>'S') and (key<>'P')  then exit;
      textcolor(white);
      if key='P' then begin
                        write('  stacja ATARI -');
                        repeat key:=upcase(any_key);until (key='A') or (key='B')
;
                        writeln(key,':');  d1:=ord(key)-65;
                        write('  stacja PC    -');
                        repeat key:=upcase(any_key);
                         until (key<>chr(65+d1)) and (key>='A') and (key<='D');
                        writeln(key,':'); xdrv:=ord(key)-64;
                        exit
                      end;
```

```
                    writeln(key,' '); xdrv:=ord(key)-64;
                    exit
                  end;
    open_win('Scanning dysku w stacji '+chr(65+disk)+':',white,black,4,7,63,19
);
    writeln(' <                                                          >');
    writeln(' < Testowanie przydatnosci dysku do przechowywania          >');
    writeln(' < danych przeznaczonych do wymiany z systemem Atari DOS >');
      write(' < ...czekam na potwierdzenie klawiszem ENTER...> ');
    key:=any_key; if ord(key)<> 13 then begin
                                          writeln('-STOP-');
                                          exit
                                        end
                                   else writeln('-START-');
    write(' <');textcolor(white+blink);write('UWAGA');textcolor(white);
    writeln(' czy dysk byl przygotowywany na ATARI (Tak/Nie) ? >');

    repeat
      key:=any_key;
      key:=upcase(key);
    until (key='T') or (key='N'); if key='T' then writeln('-TAK-') else writel
n('-NIE-');
    if key='N' then begin
                    writeln(' < Przykro mi,przygotuj dysk na ATARI...>');
                    exit
                  end;

    for xxx:=0 to 719 do write('?');
    gotoxy(1,1);
    for t:=0 to 39 do begin
       for s:=1 to 18 do begin   {oznacz sektor jako dostepny dla PC...}
           buf[0]:=1;
                                 {BOOT sektory tylko przeczytaj...}
           if (t=0) and (s<4) then bios_stat:=sec_atr(read_sec,disk,s)
                             else bios_stat:=sec_atr(write_sec,disk,t*18+s);
           if bios_stat=ok then textbackground(green)
                           else textbackground(red);
           if bios_stat=ok then write('!') else write('?')
         end;
    end;
    textbackground(black);
    write(' < Przenies TEN dysk na ATARI-koniec testowania... >');
    key:=any_key;
  end;
{-------------------------------------------------------------------------------}
  procedure comm(on:boolean;xpos:integer);
    begin
      if on then textcolor(white+blink);
      gotoxy(xpos*18+1,1);
      case xpos of
        0:write('Transfer z PC');
        1:write('Transfer z ATARI');
        2:write ('Skasuj pliki ATARI');
        3:write(' Operacje specjalne');
      end;
      textcolor(white);  gotoxy(xpos*18+1,1);
    end;
{-------------------------------------------------------------------------------}
  function menu(var nc:integer):boolean;
    var
      dc,xc:integer;
    begin
      textattr:=lightgray*16+white;window(2,2,78,6);clrscr;
      textbackground(red); textcolor(white);
      window(1,1,80,25);  gotoxy(1,25);
      write(' ENTER-wybierz opcje:ESC-koniec pracy z programem ');
      textbackground(lightgray);write('    ');textbackground(red);
      write('* ',chr(65+d1),':-stacja ATARI  ',chr(64+xdrv),':-PC *');
      open_win('',white,red,2,2,78,2);  window(3,3,77,3);
```

```
      write('* ',chr(65+d1),':-stacja ATARI  ',chr(64+xdrv),':-PC *');
      open_win('',white,red,2,2,78,2);  window(3,3,77,3);
        for xc:=0 to 3 do comm(false,xc);
      dc:=0; menu:=false;
      repeat
        if dc<>0 then comm(false,nc-dc);
        comm(true,nc);key:=any_key; dc:=0;
        if ord(key)=27 then begin
                              open_win('Koniec ? ...',white,red,2,2,78,2);
                              window(1,1,80,25);gotoxy(1,25);
                              write(' ENTER-koniec pracy z programem:ESC-anulu
j polecenie ');
                              textbackground(lightgray);write('
          ');
                              gotoxy(1,25);key:=any_key;if ord(key)<>13 then m
enu:=true;
                              exit
                            end;
        if ord(key)=0 then begin
                             key:=readkey;
                             if ord(key)=77 then if nc<3 then dc:=1;
                             if ord(key)=75 then if nc>0 then dc:=-1;
                           end;
        nc:=nc+dc;
      until ord(key)=13;
      menu:=true;              {wroc do menu po "execu"}
      textbackground(lightgray); window(1,7,80,25); clrscr;
      case nc of               {"wystartuj" proces...}
        0:begin
            pc2atr;
            window(1,1,80,25);put_logo(false,xl);
          end;
        1:begin
            if not load_dir(d1) then exit;  {nie odczytano poprawnie katalog
u ATARI}
            atr2pc;           {...dokonaj transferu na stacje XDRV:}
            window(1,1,80,25);put_logo(false,pc);
          end;
        2:begin
            if not load_dir(d1) then exit;  {nie odczytano poprawnie katalog
u ATARI}
            win_del;
            window(1,1,80,25);put_logo(false,xl);
          end;
        3:scan_disk(d1);          {wymien stacje,scan_disk,itd.}

      end;
    end;
{=============================================================================}
  begin
{demo okienek}
    textmode(c80);
    d1:=0; xdrv:=2;          {stacja "ATARI" - drive A: IBM- drive B:}

{ init_disk KONIECZNIE wywolany na poczatku programu...}
{   bufor sektora fiz. przeniesiony do obszaru zmiennych dynamicznych aby }
{   uniknac przekroczenia  granicy 64 kB !}
     if init_disk(d1)<>ok  then halt;
    win_ini(white,lightgray); title(7);  {screen tytulowy na 7 sekund...}
    ncom:=0 ;
    repeat until not menu(ncom);      {win menu....}
    win_ini(lightgray,black);
{pokaz VTOC: for xxx:=1 to $2d0 do if avail(xxx) then write('o') else write('*')
;}
  end.
```